AMERICAN INTERNATIONAL UNIVERSITY – BANGLADESH (AIUB)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE TITLE

INTRODUCTION TO DATA SCIENCE

PROJECT TITLE

FC 26 Player Data: Cleaning, Analysis & Prediction Prep

COURSE INSTRUCTOR KAMRUN NAHAR KOLI

SECTION – K (GROUP 2)

NAME	ID	CONTRIBUTION
Md. Towhidul Islam	23-55036-3	Normalization, Balancing, Train/Test Split, Introduction
Abrar Kabir	23-55095-3	Noisy Values, Outliers, Duplicates, Conclusion
Tasnim Hassan Ahona	23-55085-3	EDA, Visualization, Descriptive Statistics
Anika Tabassum	23-55070-3	Data Column Selection, Import, Missing Values, Group Comparison

Introduction & Dataset Description:

The dataset employed in this project is the FC 26 (EA Sports FC 2026) Player Dataset, which has the specifics of player features, performance scores, financial data, and demographic data of professional football players in leagues worldwide. The dataset has been extensively employed in the sport analytics studies to evaluate players, predict their performance and in the exploration of data. The dataset is rich with player records, which makes it a good source to conduct data cleaning, exploration data analysis and machine learning-based prediction activities.

Dataset Reference: Kaggle. (2025). *FC 26 Player Dataset*. Kaggle. https://www.kaggle.com/datasets/rovnez/fc-26-fifa-26-player-data/data

Selected Attributes:

1. **short_name (Player Name):** A character variable representing the short display name of the player.
2. **player_positions:** A categorical variable indicating the playing position(s) of the player on the field.
3. **overall:** Continuous numerical variable representing the player's current overall performance rating on a scale of 0–99.
4. **potential:** Continuous numerical variable representing the player's maximum projected rating, reflecting future growth capacity.
5. **value_eur (Market Value in EUR):** Continuous variable showing the estimated transfer market value of the player in Euros.
6. **wage_eur (Weekly Wage in EUR):** Continuous variable indicating the player's weekly wage in Euros; higher wages typically reflect better performing players.
7. **age:** Continuous numerical variable representing the player's age in years.
8. **height_cm (Height in cm):** Continuous variable measuring the player's height in centimeters; relevant for position specific physical analysis.
9. **weight_kg (Weight in kg):** Continuous variable measuring the player's body weight in kilograms; important for physical profiling.
10. **preferred_foot:** Binary categorical variable indicating the player's dominant foot.
    - Right = Right footed
    - Left = Left footed
11. **pace:** Continuous variable reflecting the player's speed attributes; commonly used in evaluating attackers and wing players.

12. **shooting:** Continuous variable measuring the player's shooting ability; commonly used in forward player analysis.
13. **passing:** Continuous variable representing the player's passing accuracy and vision; key for midfield evaluation.
14. **dribbling:** Continuous variable measuring the player's ball control and dribbling skill; typically elevated in attacking players.
15. **defending:** Continuous variable indicating the player's defensive capability; high values are typical for center backs and defensive midfielders.
16. **physic:** Continuous variable showing total physical attributes including strength and stamina, reflecting the player's physical presence on the field.

Data Import and View

This will load the CSV file into R with correct column headers and delimiting settings and open it in the RStudio data viewer.

Code:

```
df <- read.csv("C:\\Users\\towhi\\Downloads\\archive (3)\\FC26_20250921.csv")

df <- df[, c("short_name", "player_positions", "overall", "potential",
             "value_eur", "wage_eur", "age", "height_cm", "weight_kg",
             "preferred_foot", "pace", "shooting", "passing",
             "dribbling", "defending", "physic")]
view(df)
```

Output:
The output shows the dataset loaded successfully.

Filter

	short_name	player_positions	overall	potential	value_eur	wage_eur
1	J. Bellingham	CAM, CM	90	94	174500000	320000
2	F. Valverde	CM, CDM, RB	89	90	120500000	340000
3	J. Kimmich	CDM, RB, CM	89	89	86000000	140000
4	A. Hakimi	RB, RM	89	90	111000000	170000
5	N. Barella	CM	87	87	79500000	69000
6	H. Çalhanoğlu	CDM, CM	86	86	47500000	61000
7	F. Dimarco	LB, LM	85	85	52000000	54000
8	A. Mac Allister	CM, CDM	87	88	93000000	180000
9	Rodri	CDM, CM	90	90	102000000	270000
10	A. Griezmann	ST, LM, CAM	85	85	26000000	115000
11	T. Hernández	LB, LM	84	85	44500000	65000
12	Bruno Fernandes	CAM, CM	87	87	77000000	165000
13	J. Alvarez	ST	87	90	107000000	125000
14	T. Alexander-Arnold	RB, RM	86	87	73000000	250000
15	Sancet	CAM, CM, ST	84	88	58500000	66000

Showing 1 to 15 of 18,405 entries, 16 total columns

wage_eur	age	height_cm	weight_kg	preferred_foot	pace	shooting	passing	dribbling	defending	physic
320000	22	186	75	Right	80	86	83	90	78	85
340000	26	182	74	Right	88	84	84	84	83	85
140000	30	177	75	Right	72	74	89	84	83	79
170000	26	181	73	Right	92	79	82	83	82	79
69000	28	175	68	Right	80	78	84	86	81	76
61000	31	178	69	Right	71	81	87	82	81	73
54000	27	175	76	Left	80	78	84	82	79	77
180000	26	176	72	Right	66	82	85	85	78	76
270000	29	190	82	Right	65	80	86	84	86	85
115000	34	176	67	Left	77	86	85	88	59	73
65000	27	184	81	Left	90	76	78	83	79	83
165000	30	179	69	Right	67	83	89	83	65	75
125000	25	170	71	Right	85	87	81	87	57	79
250000	26	180	69	Right	76	72	89	80	80	74
66000	25	188	77	Right	76	83	80	85	70	82
71000	30	191	80	Left	81	79	81	80	77	84
100000	31	180	70	Right	75	78	83	83	76	83
175000	25	181	79	Right	79	74	82	80	81	83
95000	30	184	74	Right	89	79	78	81	78	82
78000	30	189	82	Right	77	78	80	80	80	82
170000	25	172	64	Right	72	80	86	90	75	70
370000	33	175	72	Left	89	88	86	90	45	76
52000	31	178	76	Right	74	80	80	80	76	76
210000	26	185	80	Right	72	73	84	80	83	83
68000	31	182	74	Right	83	73	84	84	78	74
170000	28	181	74	Right	82	71	85	87	78	77

Dataset Dimensions:

Code:

```
cat("Number of rows:", nrow(data), "\n")
cat("Number of columns:", ncol(data))
```

Output:

```
> cat("Number of rows:", nrow(df), "\n")
Number of rows: 18405
> cat("Number of columns:", ncol(df))
Number of columns: 16
>
```

The output indicates that the dataset has 18,405 records of players and 110 columns that store different player attributes. There are too many columns (110) to analyze focusedly, and therefore, we will choose only 16 key variables that are pertinent to our project.

Dataset Structure:

Code:

```
str(df)
```

Output:
The str() function provides the internal structure of the dataframe. The output lists each variable name, its data type (character, numeric, factor), and example values.

```
> str(df)
'data.frame':   18405 obs. of  16 variables:
 $ short_name      : chr  "J. Bellingham" "F. Valverde" "J. Kimmich" "A. Hakimi"
...
 $ player_positions: chr  "CAM, CM" "CM, CDM, RB" "CDM, RB, CM" "RB, RM" ...
 $ overall         : int  90 89 89 89 87 86 85 87 90 85 ...
 $ potential       : int  94 90 89 90 87 86 85 88 90 85 ...
 $ value_eur       : int  174500000 120500000 86000000 111000000 79500000 47500000
52000000 93000000 102000000 26000000 ...
 $ wage_eur        : int  320000 340000 140000 170000 69000 61000 54000 180000 270
000 115000 ...
 $ age             : int  22 26 30 26 28 31 27 26 29 34 ...
 $ height_cm       : int  186 182 177 181 175 178 175 176 190 176 ...
 $ weight_kg       : int  75 74 75 73 68 69 76 72 82 67 ...
 $ preferred_foot  : chr  "Right" "Right" "Right" "Right" ...
 $ pace            : int  80 88 72 92 80 71 80 66 65 77 ...
 $ shooting        : int  86 84 74 79 78 81 78 82 80 86 ...
 $ passing         : int  83 84 89 82 84 87 84 85 86 85 ...
 $ dribbling       : int  90 84 84 83 86 82 82 85 84 88 ...
 $ defending       : int  78 83 83 82 81 81 79 78 86 59 ...
 $ physic          : int  85 85 79 79 76 73 77 76 85 73 ...
>
```

Introducing Intentional Data Issues:

The following code intentionally introduces missing values, noisy/invalid values, and extreme outliers into the dataset to simulate real world data challenges.

1. **Missing Values:**

Code:

```
df$age[100] <- NA
df$height_cm[300] <- NA
df$weight_kg[400] <- NA
df$overall[500] <- NA
df$potential[800] <- NA
```

Output:

```
> df$age[100]
[1] 30
> df$age[100] <- NA
> df$age[100]
[1] NA
>
> df$height_cm[300]
[1] 175
> df$height_cm[300] <- NA
> df$height_cm[300]
[1] NA
>
> df$weight_kg[400]
[1] 79
> df$weight_kg[400] <- NA
> df$weight_kg[400]
[1] NA
>
> df$overall[500]
[1] 74
> df$overall[500] <- NA
> df$overall[500]
[1] NA
>
> df$potential[800]
[1] 79
> df$potential[800] <- NA
> df$potential[800]
[1] NA
```

The output confirms that each value was successfully replaced with NA.

2. **Noisy or Invalid Values**

Code:

```
df$age[350] <- 999
df$value_eur[8000] <- -99999
df$physic[1100] <- -10
df$preferred_foot[200] <- "Hand"
df$preferred_foot[3000] <- "none"
df$height_cm[450] <- 999
df$shooting[5500] <- -50
df$weight_kg[12000] <- -30
df$passing[7777] <- 999
df$age[16000] <- -12
```

Output:

```
> df$age[350]
[1] 31
> df$age[350] <- 999
> df$age[350]
[1] 999
>
> df$value_eur[8000]
[1] 1800000
> df$value_eur[8000] <- -99999
> df$value_eur[8000]
[1] -99999
>
> df$physic[1100]
[1] 80
> df$physic[1100] <- -10
> df$physic[1100]
[1] -10
>
> df$preferred_foot[200]
[1] "Right"
> df$preferred_foot[200] <- "Hand"
> df$preferred_foot[200]
[1] "Hand"
>
> df$preferred_foot[3000]
[1] "Right"
> df$preferred_foot[3000] <- "none"
> df$preferred_foot[3000]
[1] "none"
```

```
> df$height_cm[450]
[1] 191
> df$height_cm[450] <- 999
> df$height_cm[450]
[1] 999
>
> df$shooting[5500]
[1] 54
> df$shooting[5500] <- -50
> df$shooting[5500]
[1] -50
>
> df$weight_kg[12000]
[1] 69
> df$weight_kg[12000] <- -30
> df$weight_kg[12000]
[1] -30
>
> df$passing[7777]
[1] 58
> df$passing[7777] <- 999
> df$passing[7777]
[1] 999
>
> df$age[16000]
[1] 32
> df$age[16000] <- -12
> df$age[16000]
[1] -12
```

The output shows the original value of each row before it was changed and the new invalid value after it has been changed. In fact, the age of Row 350 was altered to 999 which is not humanly possible as compared to its original realistic value.

3. **Outliers Inserted**

Code:

```
df$wage_eur[900] <- 9999999
df$value_eur[4500] <- 750000000
df$wage_eur[11000] <- 7500000
df$weight_kg[6600] <- 350
df$potential[2200] <- 250
df$height_cm[14500] <- 15
df$value_eur[17000] <- 650000000
```

Output:

```
> df$wage_eur[900]
[1] 31000
> df$wage_eur[900] <- 9999999
> df$wage_eur[900]
[1] 9999999
>
> df$value_eur[4500]
[1] 2100000
> df$value_eur[4500] <- 750000000
> df$value_eur[4500]
[1] 7.5e+08
>
> df$wage_eur[11000]
[1] 3000
> df$wage_eur[11000] <- 7500000
> df$wage_eur[11000]
[1] 7500000
>
> df$weight_kg[6600]
[1] 77
> df$weight_kg[6600] <- 350
> df$weight_kg[6600]
[1] 350
>
> df$potential[2200]
[1] 77
> df$potential[2200] <- 250
> df$potential[2200]
[1] 250
```

```
> df$height_cm[14500]
[1] 196
> df$height_cm[14500] <- 15
> df$height_cm[14500]
[1] 15
>
> df$value_eur[17000]
[1] 525000
> df$value_eur[17000] <- 650000000
> df$value_eur[17000]
[1] 6.5e+08
```

The output shows the original values of each row with the outlier inserted. As an illustration, the weekly wage of Row 900 was overstated to 9,999,999 EUR, which is much higher than any conceivable player salary.

4. **Duplicate Rows Inserted**

Code:

```
cat("Rows before duplicates:", nrow(df), "\n")
df <- rbind(df, df[10, ])
df <- rbind(df, df[25, ])
df <- rbind(df, df[50, ])
df <- rbind(df, df[130, ])
cat("Rows after duplicates:", nrow(df), "\n")
```

Output:

```
> cat("Rows before duplicates:", nrow(df), "\n")
Rows before duplicates: 18405
> df <- rbind(df, df[10, ])
> df <- rbind(df, df[25, ])
> df <- rbind(df, df[50, ])
> df <- rbind(df, df[130, ])
> cat("Rows after duplicates:", nrow(df), "\n")
Rows after duplicates: 18409
```

The output shows the row count before and after inserting duplicates. Four exact copies of existing rows (10, 25, 50, and 130) were appended to the end of the dataset, increasing the total row count by 4.

Handling Missing Values:

1. **Identifying Missing Values**

This code uses colSums(is.na()) to count missing values in each column. It helps identify which variables contain NA entries that require cleaning.

Code:

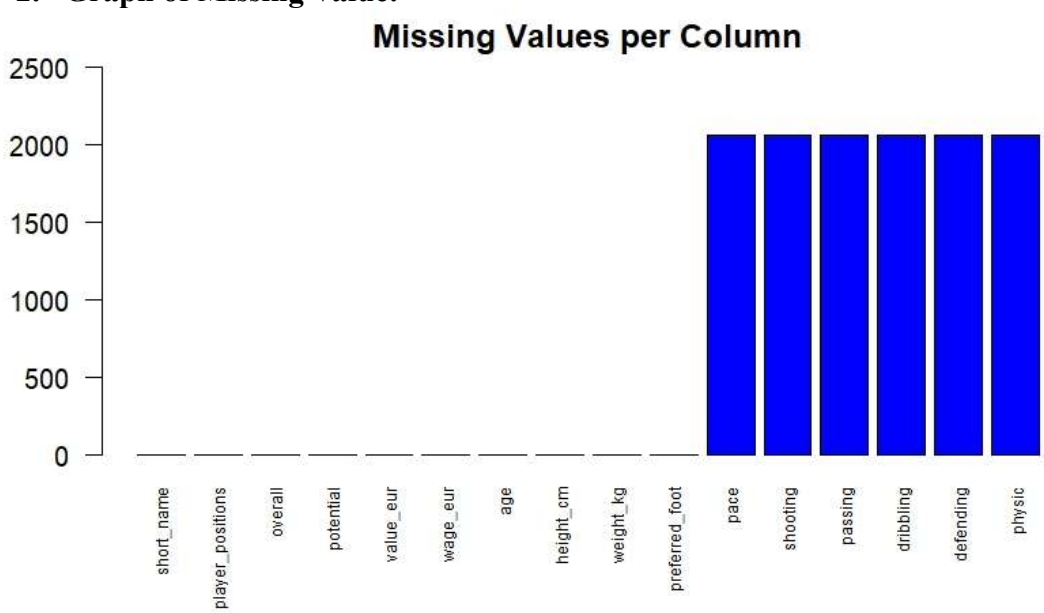
colSums(is.na(df))

Output:

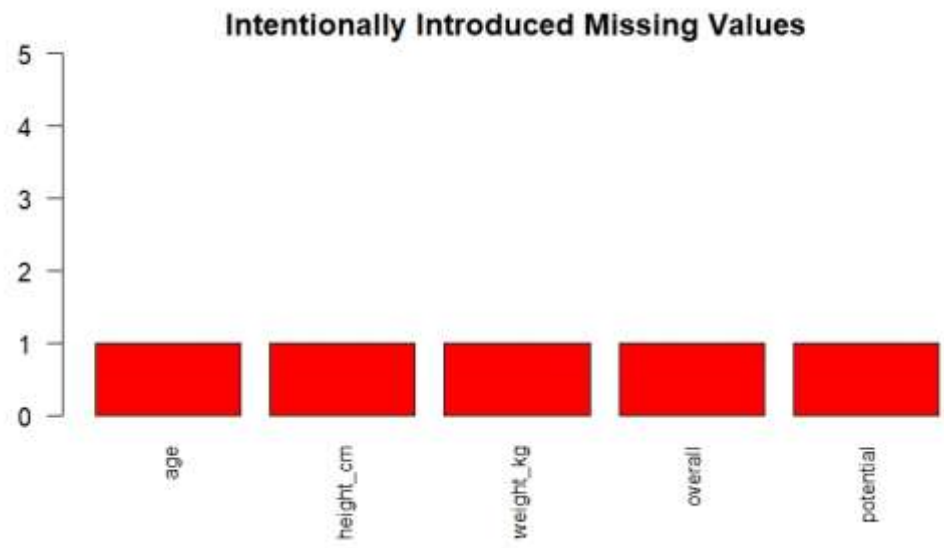
The result contains a numerical number of the number of missing values per column. In the age, height cm, weight kg, overall and potential columns (1 each) five missing values were deliberately inserted. Also, there are about 2062 natural missing values in the pace, shooting, passing, dribbling, defending and physic columns of which they belong to goalkeeper players who lack outfield attribute ratings in FIFA dataset.

```
> colSums(is.na(df))
      short_name player_positions          overall        potential
               0               0                1                1
       value_eur         wage_eur              age        height_cm
               0               0                1                1
       weight_kg   preferred_foot             pace         shooting
               1               0             2062             2062
         passing        dribbling        defending           physic
            2062            2062             2062             2062
> |
```

2. **Graph of Missing Value:**



All columns



Only intentional missing values

3. **Replacing Missing Values**

Code:

With Mean:

```
df$age[is.na(df$age)] <- mean(df$age, na.rm = TRUE)
df$height_cm[is.na(df$height_cm)] <- mean(df$height_cm, na.rm = TRUE)
df$weight_kg[is.na(df$weight_kg)] <- mean(df$weight_kg, na.rm = TRUE)
df$overall[is.na(df$overall)] <- mean(df$overall, na.rm = TRUE)
df$potential[is.na(df$potential)] <- mean(df$potential, na.rm = TRUE)
```

With Median:

```
df$pace[is.na(df$pace)] <- median(df$pace, na.rm = TRUE)
df$shooting[is.na(df$shooting)] <- median(df$shooting, na.rm = TRUE)
df$passing[is.na(df$passing)] <- median(df$passing, na.rm = TRUE)
df$dribbling[is.na(df$dribbling)] <- median(df$dribbling, na.rm = TRUE)
df$defending[is.na(df$defending)] <- median(df$defending, na.rm = TRUE)
df$physic[is.na(df$physic)] <- median(df$physic, na.rm = TRUE)
```

Verify:

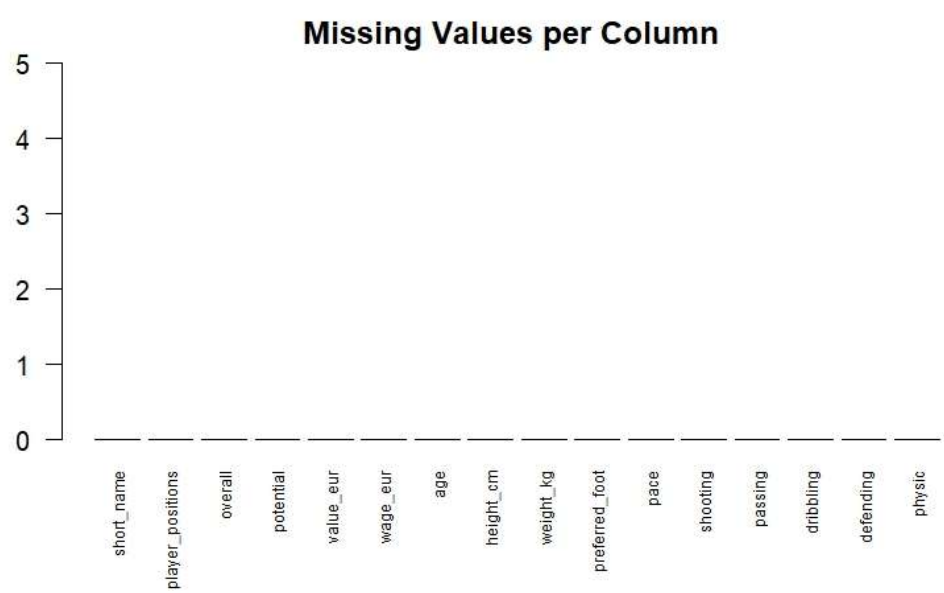
```
colSums(is.na(df))
```

Output:

```
> colSums(is.na(df))
      short_name player_positions          overall
               0                0                0
       potential        value_eur         wage_eur
               0                0                0
             age        height_cm        weight_kg
               0                0                0
  preferred_foot             pace         shooting
               0                0                0
         passing        dribbling        defending
               0                0                0
          physic
               0
>
```

The 5 deliberately added missing values (age, height_cm, weight_kg, overall, potential) were mean imputed as the columns are approximately normally distributed with the mean giving a good central tendency measure. The natural goalkeeper missing values (pace, shooting, passing, dribbling, defending, physic) were imputed with median, as these columns can have skewed distributions, and the median is less susceptible to skewness as it is not affected by extreme values.

Graph After Removing Missing Values:



Verifying the changed values:

df$age[100]
df$height_cm[300]
df$weight_kg[400]
df$overall[500]
df$potential[800]

Output:

```
> df$age[100]
[1] 25.27336
> df$height_cm[300]
[1] 182.0351
> df$weight_kg[400]
[1] 75.1508
> df$overall[500]
[1] 65.77021
> df$potential[800]
[1] 71.17693
```

The output shows the imputed values of the 5 deliberately missing rows. Row 100's age was replaced with the mean value 25.27, Row 300's height was replaced with 182.04 cm, Row 400's weight with 75.15 kg, Row 500's overall rating with 65.77, and Row 800's potential with 71.18. These values are the averages of the columns, which proves that the mean imputation was used properly.

Handling Noisy Values:

Noisy values are the ones which are either wrong or unrealistic such as impossible numbers or wrong categories. Noisy values were adjusted by being substituted with a more plausible value. The mode and median were selected in the case of numeric variables since it is not affected by extreme values. The most frequent category (mode) was used in the case of categorical variables.

Code:

```
df$age[350] <- median(df$age, na.rm = TRUE)
df$age[16000] <- mean(df$age, na.rm = TRUE)
df$value_eur[8000] <- mean(df$value_eur, na.rm = TRUE)
df$physic[1100] <- median(df$physic, na.rm = TRUE)
df$preferred_foot[200] <- "Right"
df$preferred_foot[3000] <- "Left"
df$height_cm[450] <- mean(df$height_cm, na.rm = TRUE)
df$shooting[5500] <- median(df$shooting, na.rm = TRUE)
df$weight_kg[12000] <- mean(df$weight_kg, na.rm = TRUE)
df$passing[7777] <- median(df$passing, na.rm = TRUE)
```

Output:

```
> df$age[350]
[1] 999
> df$age[350] <- median(df$age, na.rm = TRUE)
> df$age[350]
[1] 25
> df$age[16000]
[1] -12
> df$age[16000] <- mean(df$age, na.rm = TRUE)
> df$age[16000]
[1] 25.22045
>
> df$value_eur[8000]
[1] -99999
> df$value_eur[8000] <- mean(df$value_eur, na.rm = TRUE)
> df$value_eur[8000]
[1] 3013292
>
> df$physic[1100]
[1] -10
> df$physic[1100] <- median(df$physic, na.rm = TRUE)
> df$physic[1100]
[1] 66
```

```
> df$preferred_foot[200]
[1] "Hand"
> df$preferred_foot[200] <- "Right"
> df$preferred_foot[200]
[1] "Right"
>
> df$preferred_foot[3000]
[1] "none"
> df$preferred_foot[3000] <- "Left"
> df$preferred_foot[3000]
[1] "Left"
>
>
> df$height_cm[450]
[1] 999
> df$height_cm[450] <- mean(df$height_cm, na.rm = TRUE)
> df$height_cm[450]
[1] 182.0351
>
> df$shooting[5500]
[1] -50
> df$shooting[5500] <- median(df$shooting, na.rm = TRUE)
> df$shooting[5500]
[1] 55
```

```
>
> df$weight_kg[12000]
[1] -30
> df$weight_kg[12000] <- mean(df$weight_kg, na.rm = TRUE)
> df$weight_kg[12000]
[1] 75.1508
>
>
> df$passing[7777]
[1] 999
> df$passing[7777] <- median(df$passing, na.rm = TRUE)
> df$passing[7777]
[1] 58
>
```

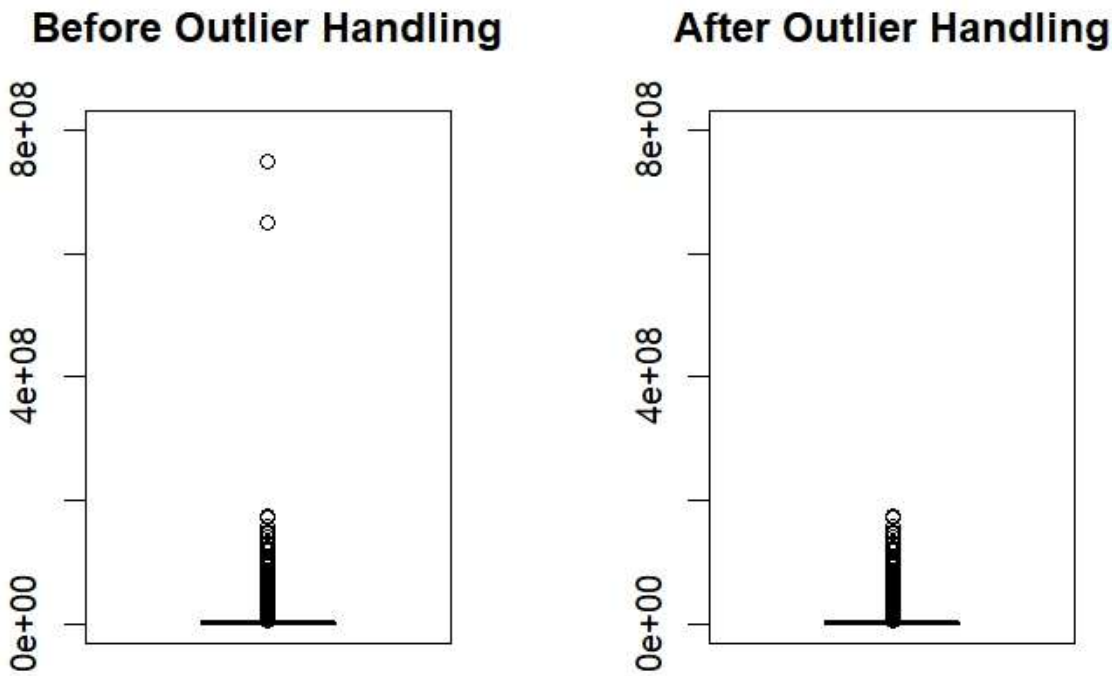
The output indicates that all 10 noisy values were corrected. Invalid numeric values like 999, -12, -99999, -10, -50, -30, and 999 were replaced with either the mean or median of their respective columns. In the case of categorical entries, Hand was changed to Right and none to Left. It was done in three ways, mean when dealing with symmetric columns, median when dealing with skewed columns and mode replacement when dealing with categorical columns.

Handling Outliers:

For wage
Code:

```
boxplot(df$wage_eur, main = "wage_eur - Before Outlier Handling")
df$wage_eur[900]
df$wage_eur[900] <- median(df$wage_eur, na.rm = TRUE)
df$wage_eur[900]
df$wage_eur[11000]
df$wage_eur[11000] <- median(df$wage_eur, na.rm = TRUE)
df$wage_eur[11000]
boxplot(df$wage_eur, main = "wage_eur - After Outlier Handling")
```

Output:



The boxplots depict the value of the eur prior to and after the outliers. The Before plot indicates that the y-axis extends to 600 million EUR because the two extreme outliers (750,000,000 and 650,000, 000) have been carefully placed at row 4500 and 17000. Once these are substituted with the median value, the "After" plot has the y-axis scaled down to a much smaller 150 million EUR.

For Weight:

Code:

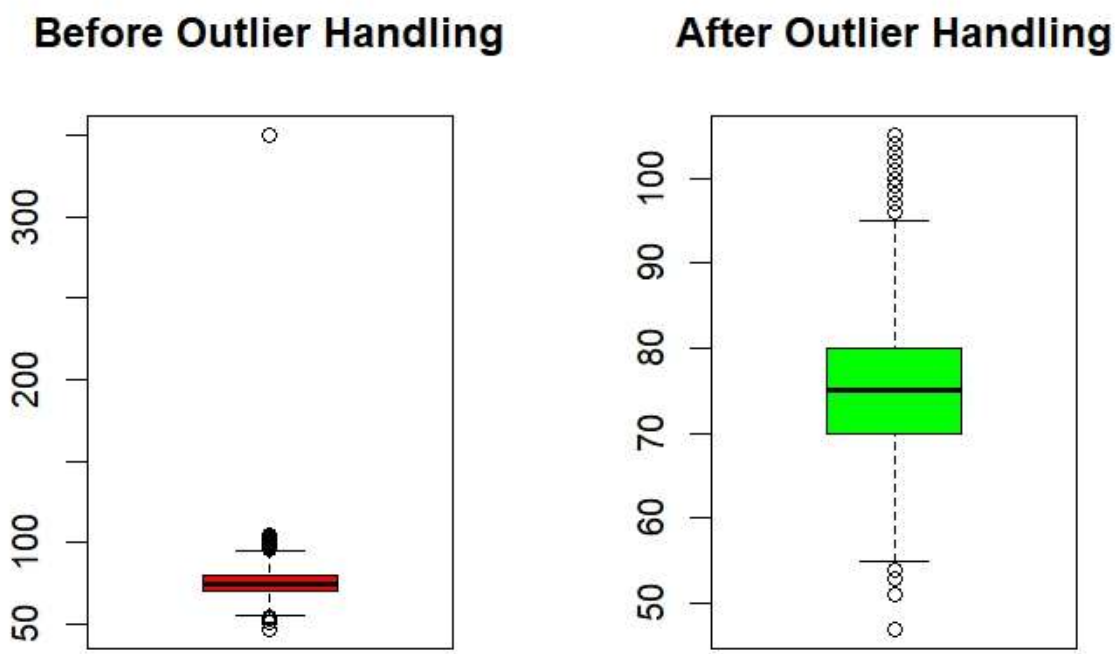
```
par(mfrow = c(1, 2))

df$weight_kg[6600] <- 350
boxplot(df$weight_kg, main = "Before Outlier Handling", col = "red")

df$weight_kg[6600] <- median(df$weight_kg, na.rm = TRUE)
boxplot(df$weight_kg, main = "After Outlier Handling", col = "green")

par(mfrow = c(1, 1))
```

Output:



The boxplots reveal the weight kg prior to through outlier handling. The "Before" plot shows the y-axis stretching up to 350 kg due to the intentionally inserted extreme outlier at row 6600. The bottom of the box is compressed and it is hard to see the actual data distribution. Once the outlier has been replaced with the median value, the After plot provides a clear picture of the actual distribution of player weights that lie between about 50 and 100 kg with the median of 75 kg.

Potential outlier:

Code:

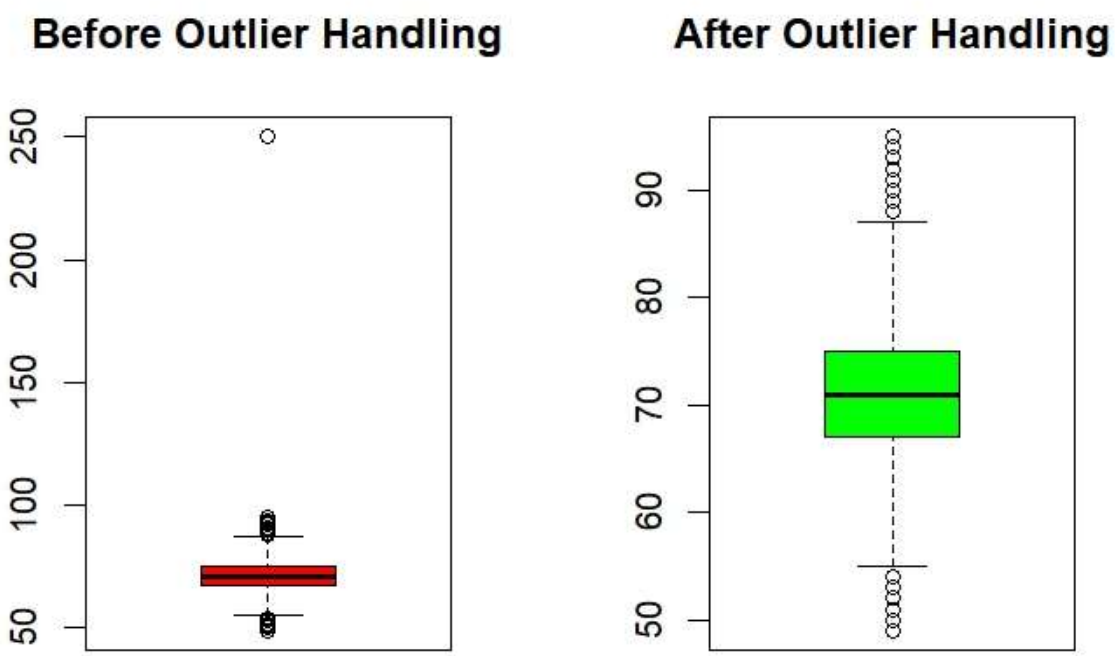
```
dev.off()
par(mfrow = c(1, 2))

df$potential[2200] <- 250
boxplot(df$potential, main = "Before Outlier Handling", col = "red")

df$potential[2200] <- median(df$potential, na.rm = TRUE)
boxplot(df$potential, main = "After Outlier Handling", col = "green")

par(mfrow = c(1, 1))
```

Output:



The boxplots indicate a possible pre-and post-outlier treatment. The Before plot demonstrates the y-axis having a value of 250 as the outlier at row 2200 is deliberately created and thus it is much higher than the maximum possible rating of 99. The box is compressed at the bottom making the distribution unreadable. Following the substitution of the outlier with the median, the After plot depicts the actual shape of the players potential ratings that show a range of about 50 to 95 with the median of about 71.

height_cm outlier:

Code:

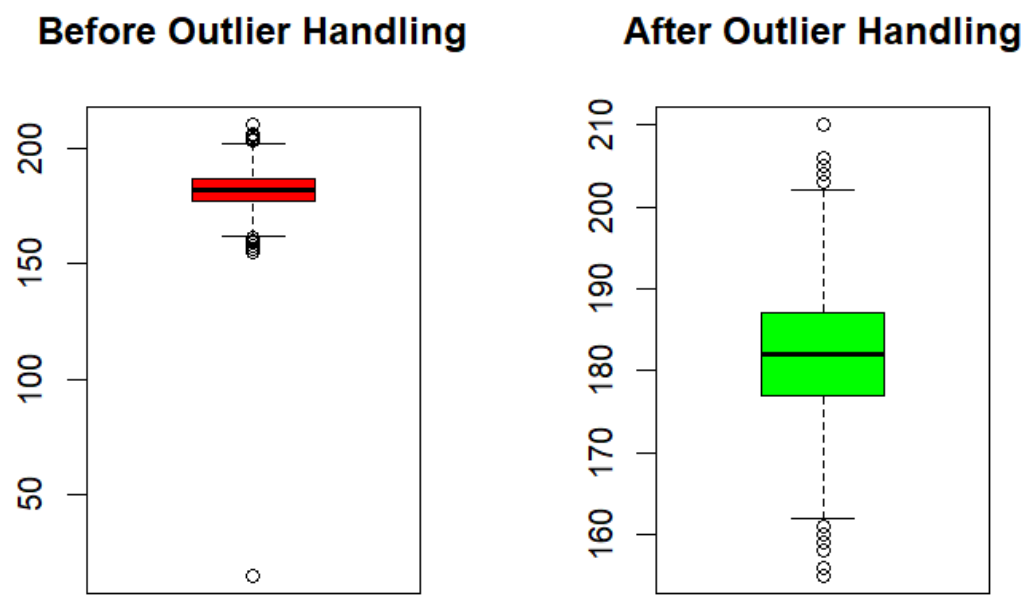
```
dev.off()
par(mfrow = c(1, 2))

df$height_cm[14500] <- 15
boxplot(df$height_cm, main = "Before Outlier Handling", col = "red")

df$height_cm[14500] <- median(df$height_cm, na.rm = TRUE)
boxplot(df$height_cm, main = "After Outlier Handling", col = "green")

par(mfrow = c(1, 1))
```

Output:



The boxplots reveal the height -1cm and after outlier handling. The Before plot depicts one point at the bottom with the unrealistically added height of 15 cm at row 14500, the Before plot. This very low value causes the y-axis to be stretched down to almost 0, therefore compressing the actual distribution of data. Once the outlier has been replaced with the average, the "After" plot shows clearly the actual distribution of the player heights with a range of between approximately 160 to 210 cm and a median of about 182 cm which are realistic heights of professional football players.

Handling Duplicates:

Code:

```
cat("Rows before removing duplicates:", nrow(df), "\n")
sum(duplicated(df))

df <- df[!duplicated(df), ]
cat("Rows after removing duplicates:", nrow(df), "\n")
sum(duplicated(df))
```

Output:

```
> cat("Rows before removing duplicates:", nrow(df), "\n")
Rows before removing duplicates: 18409
> sum(duplicated(df))
[1] 4
>
> df <- df[!duplicated(df), ]
> cat("Rows after removing duplicates:", nrow(df), "\n")
Rows after removing duplicates: 18405
> sum(duplicated(df))
[1] 0
>
```

The duplicated() function was employed to identify the exact same duplicate rows in the data set. On purpose, four duplicate rows (copy of rows 10, 25, 50, and 130) had been inserted. sum(duplicated) gave confirmation that there were duplicate entries in the table. Following the application of the removal with df[!duplicated(df),] all the duplicate rows were removed successfully. The number of rows reduced to 18,405 as opposed to 18,409, which proves that all the 4 purposeful duplicates were identified and removed.

Normalizing the Dataset:

A min-max normalization function scales selected columns to a 0–1 range: (x - min) / (max - min). The normalized dataset is stored separately to preserve the original.

Code:

```
normalize_dataset <- df
normalize <- function(x) {return((x - min(x)) / (max(x) - min(x)))}
normalize_dataset$age <- normalize(df$age)
normalize_dataset$overall <- normalize(df$overall)
head(normalize_dataset[, c("short_name", "age", "overall")])
```

Output:

```
> normalize_dataset <- df
> normalize <- function(x) {return((x - min(x)) / (max(x) - min(x)))}
> normalize_dataset$age <- normalize(df$age)
> normalize_dataset$overall <- normalize(df$overall)
> head(normalize_dataset[, c("short_name", "age", "overall")])
     short_name       age   overall
1 J. Bellingham 0.2142857 0.9772727
2   F. Valverde 0.3571429 0.9545455
3    J. Kimmich 0.5000000 0.9545455
4     A. Hakimi 0.3571429 0.9545455
5    N. Barella 0.4285714 0.9090909
6 H. Çalhanoğlu 0.5357143 0.8863636
>
```

The age and overall columns were normalized using the min-max normalization function and their values were converted to the range of 0 to 1. To illustrate, the age of J. Bellingham was scaled to 0.21 (that means that he is a relatively young player) and his overall rating to 0.98 (that means that he is one of the highest-rated players). An age of J. Kimmich normalized to 0.50 (mid-range) and in general, 0.95. The normalized data was saved in a different data set as normalize_dataset to maintain the original data to be further analyzed.

Exploratory Data Analysis:

1. **Summary Statistics:**

Code:

summary(df)

Output:

```
> summary(df)
  short_name        player_positions      overall        potential       value_eur            wage_eur
 Length:18405       Length:18405       Min.   :47.00   Min.   :49.00   Min.   :        0   Min.   :     0
 Class :character   Class :character   1st Qu.:61.00   1st Qu.:67.00   1st Qu.:   475000   1st Qu.:  1000
 Mode  :character   Mode  :character   Median :66.00   Median :71.00   Median :  1000000   Median :  4000
                                       Mean   :65.77   Mean   :71.16   Mean   :  2931665   Mean   : 10095
                                       3rd Qu.:70.00   3rd Qu.:75.00   3rd Qu.:  2100000   3rd Qu.: 10000
                                       Max.   :91.00   Max.   :95.00   Max.   :174500000   Max.   :610000
      age          height_cm     weight_kg      preferred_foot          pace          shooting
 Min.   :16.00   Min.   :155   Min.   : 47.00   Length:18405       Min.   :30.00   Min.   :21.00
 1st Qu.:21.00   1st Qu.:177   1st Qu.: 70.00   Class :character   1st Qu.:63.00   1st Qu.:45.00
 Median :25.00   Median :182   Median : 75.00   Mode  :character   Median :69.00   Median :55.00
 Mean   :25.22   Mean   :182   Mean   : 75.14                      Mean   :68.44   Mean   :53.06
 3rd Qu.:29.00   3rd Qu.:187   3rd Qu.: 80.00                      3rd Qu.:75.00   3rd Qu.:62.00
 Max.   :44.00   Max.   :210   Max.   :105.00                      Max.   :97.00   Max.   :92.00
    passing        dribbling       defending         physic
 Min.   :25.00   Min.   :22.00   Min.   :15.00   Min.   :32.0
 1st Qu.:52.00   1st Qu.:58.00   1st Qu.:41.00   1st Qu.:59.0
 Median :58.00   Median :64.00   Median :57.00   Median :66.0
 Mean   :57.66   Mean   :63.01   Mean   :52.52   Mean   :64.9
 3rd Qu.:63.00   3rd Qu.:68.00   3rd Qu.:63.00   3rd Qu.:71.0
 Max.   :92.00   Max.   :93.00   Max.   :90.00   Max.   :91.0
>
```

The summary statistics will be a description of all the distribution of variables in the cleaned data. The total rating is between 47 and 91 with an average of 65.77 meaning that most players are mid-range rated. The possible potential is between 49 and 95 with an average of 71.16 indicating that players have a higher potential compared to the current rating. The age of the players is between 16 and 44 years with a median of 25. The market values are very diverse with a range of 0 to 174.5 million EUR with an average of around 2.93 million EUR and a highly right-skewed distribution with a few very high-value players. The pattern of weekly wages is similar and skewed with the highest wage of 0 to 610,000 EUR. The physical characteristics such as height (mean 182 cm) and weight (mean 75.14 kg) are within the acceptable limits of professional athletes. Skill ratings (pace, shooting, passing, dribbling, defending, physic) are all about 15-97 with means of 52-68.

More detailed statistics:

Code:

summary(df$age)
summary(df$overall)

table(df$preferred_foot)

Output:

```
> summary(df$age)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  16.00   21.00   25.00   25.22   29.00   44.00 
> summary(df$overall)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  47.00   61.00   66.00   65.77   70.00   91.00 
> 
> table(df$preferred_foot)

 Left Right 
 4498 13907 
> 
```

Ages of players are between 16 and 44 years old with median 25 years which means that the data are slightly skewed on younger players. The total score is 47-91 with median of 66 and mean of 65.77 with the majority of the players found in the 60s middle range. The distribution of preferred foot indicates a strong presence of right-footed (13,907) and left-footed (4,498) players, which is indicative of real world football statistics of about 75% of players being right-footed.

Feature Analysis: Detailed Overall Rating Statistics

Code:

```
cat("Mean Overall:", mean(df$overall), "\n")
cat("Median Overall:", median(df$overall), "\n")
cat("SD Overall:", sd(df$overall), "\n")
cat("Min Overall:", min(df$overall), "\n")
cat("Max Overall:", max(df$overall), "\n")
```

Output:

```
> cat("Mean Overall:", mean(df$overall), "\n")
Mean Overall: 65.76652
> cat("Median Overall:", median(df$overall), "\n")
Median Overall: 66
> cat("SD Overall:", sd(df$overall), "\n")
SD Overall: 6.980364
> cat("Min Overall:", min(df$overall), "\n")
Min Overall: 47
> cat("Max Overall:", max(df$overall), "\n")
Max Overall: 91
```

The comprehensive statistics of the Overall rating indicate that the mean (65.77) and median (66) are quite similar with an indication of a close to symmetric distribution. Standard deviation of 6.98 implies that there is moderate dispersion about the mean. The lowest score is 47 and highest score is 91 with a range of 44. The majority of the players lie within a single standard deviation of the mean (around 59 to 73), which is the range of average-to-good professional players. The highest rated (85 and above) are the elite footballers whereas the lowest end (less than 55) is the reserve or youth squad players.

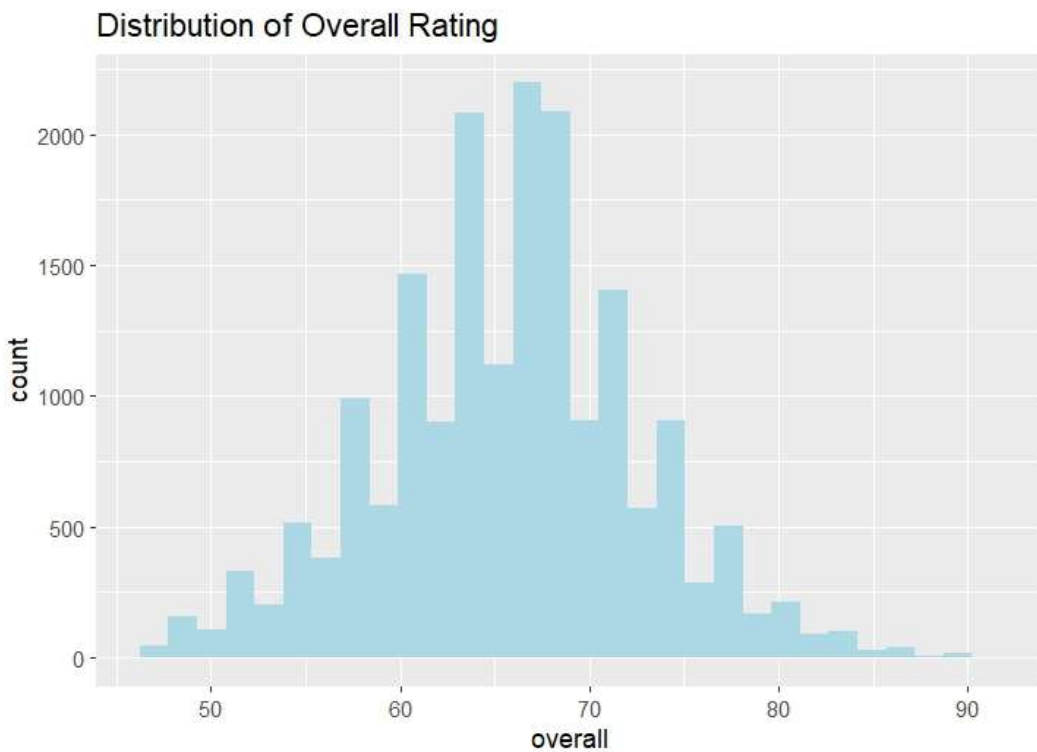
Visualizing Distributions:

1. **Histogram:**

Code:

```
ggplot(df, aes(overall)) +
  geom_histogram(bins = 30, fill = "lightblue") +
  ggtitle("Distribution of Overall Rating")
```

Output:



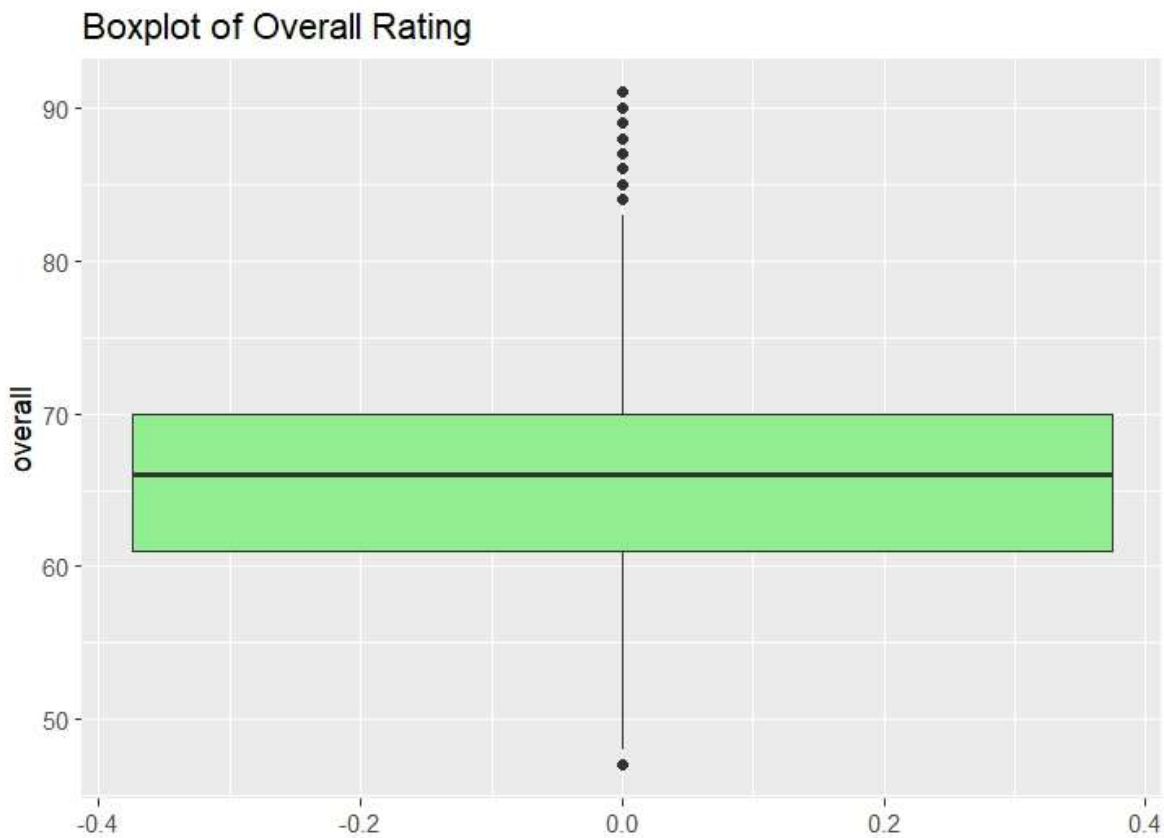
The frequency distribution of the Overall rating is presented in the histogram. This distribution is more or less bell-shaped and slightly left skewed with the most concentration around 65-68 where the majority of the players exceed 2000. Most of the players are between 58-75, which is the middle-level professional players. The left tail (less than 55) has fewer players that represent lower rated youth or reserve players whereas the right tail (greater than 80) has very few players that are of an elite player status as seen in the dataset.

2. **Boxplot:**

Code:

```
ggplot(df, aes(y = overall)) +
  geom_boxplot(fill = "lightgreen") +
  ggtitle("Boxplot of Overall Rating")
```

Output:



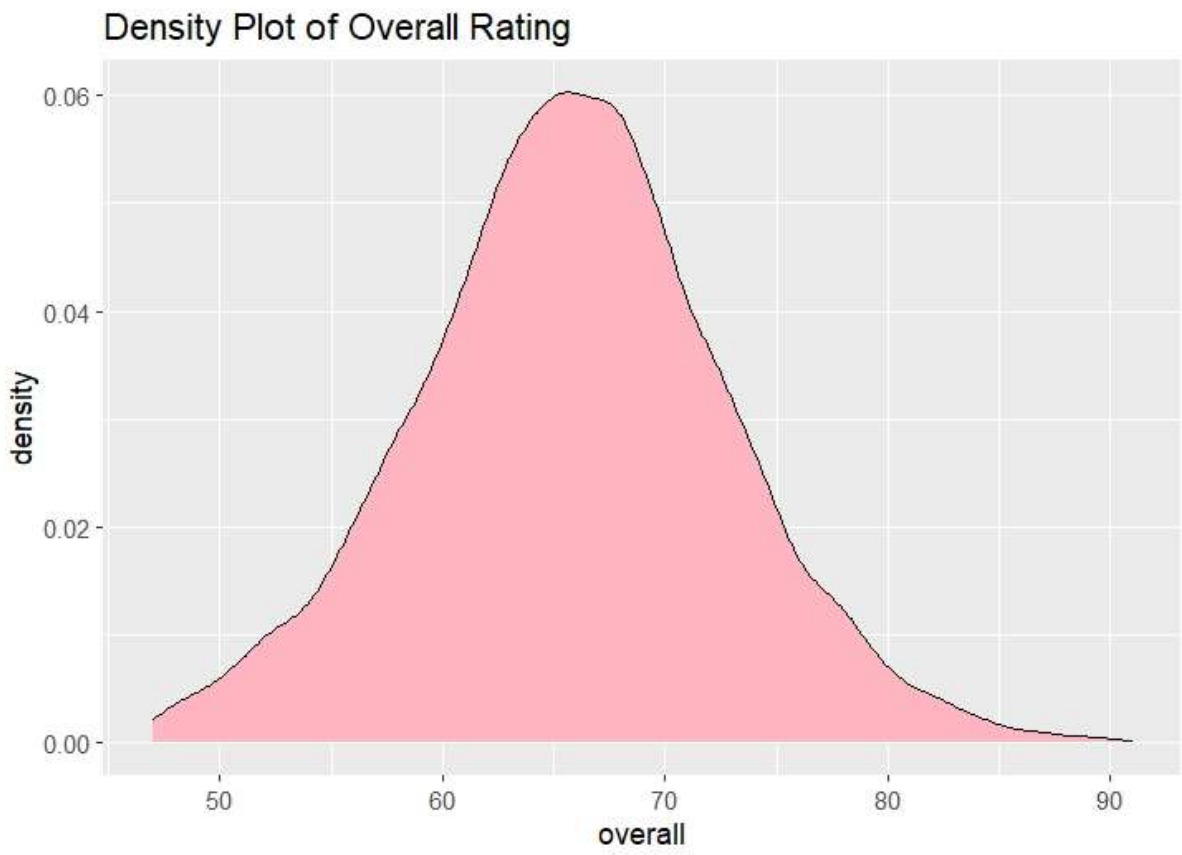
The boxplot shows how the distribution and spread of the Overall rating is. The median is around 66 with the interquartile range (IQR) of 61 (Q1) to 70 (Q3), meaning half of all the players are between 61 and 70. The lower whisker is about 48 and the upper about 82. The dots above the upper whisker indicate outliers - these are the players that are of an elite level and have a rating of 83 or higher (like Mbappé, Haaland, Bellingham). The only single dot at the bottom is that of the lowest rated player of 47. There are no extreme artificial outliers, and thus the data is clean.

3. **Density Plot:**

Code:

```
ggplot(df, aes(overall)) +
  geom_density(fill = "lightpink") +
  ggtitle("Density Plot of Overall Rating")
```

Output:



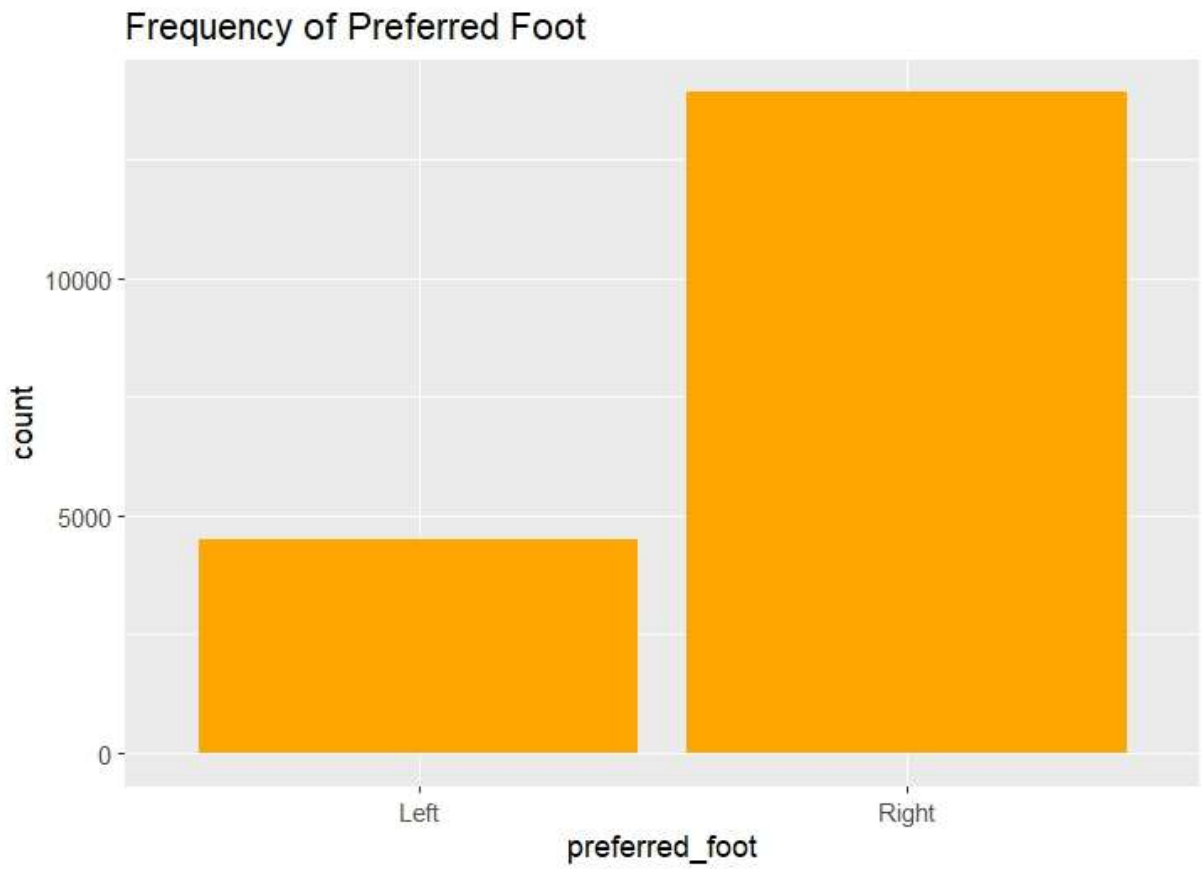
The density plot shows a smoothed probability distribution of the Overall rating. The curve shows that there is a single mode (single peak) distribution whose peak is approximately 66-68, which proves that the majority of players fall in the range. Its distribution is a bit right-skewed with a longer tail to the right with high ratings (80-91) which means that elite players are rare. The tail of the left is steeper below 50, with very few very low-rated players. The continuity of the curve proves the existence of an approximately normal distribution with a small positive skew.

Category Frequency Plot:

Code:

```
ggplot(df, aes(preferred_foot)) +
  geom_bar(fill = "orange") +
  ggtitle("Frequency of Preferred Foot")
```

Output:



The frequency distribution of the preferred foot variable is presented in the bar chart. The dataset is dominated by right-footed players making it around 13,907 players whereas left-footed players make 4,498. This 75-25 percent proportion is a true representation of the demographics of real-life football as most professional footballers are inherently right-footed. The chart also ascertains that the two invalid categories (Ambidextrous and none) that have been used above have been effectively cleaned as only Left and Right are remaining.

Group Comparison:

Code:

```
tapply(df$overall, df$preferred_foot, mean)
tapply(df$overall, df$preferred_foot, sd)
```

Output:

```
> tapply(df$overall, df$preferred_foot, mean)
    Left    Right
66.45998 65.54223
> tapply(df$overall, df$preferred_foot, sd)
    Left    Right
6.719338 7.048372
>
```

The group comparison shows that left-footed players have a slightly higher mean overall rating (66.46) than right-footed players (65.54), which is about 0.92 points higher. The standard deviation for left-footed players is 6.72, which is a little lower than the standard deviation for right-footed players, which is 7.05. This means that left-footed player ratings are a little more consistent. This small difference could mean that left-handed players who make it to the pros tend to be rated a little higher on average, but the difference isn't big enough to make a strong case.

Descriptive Statistics Across Target Classes (rating_class):

Code:

tapply(df$wage_eur, df$rating_class, mean)
tapply(df$wage_eur, df$rating_class, sd)
tapply(dfage, dfrating_class, mean)

Output:

```
> tapply(df$wage_eur, df$rating_class, mean)
Below Average       Average          Good
     1791.124      9232.814     43260.328
        Elite
   150452.632
> tapply(df$wage_eur, df$rating_class, sd)
Below Average       Average          Good
     1822.533      8544.536     30858.188
        Elite
    87341.561
> tapply(df$age, df$rating_class, mean)
Below Average       Average          Good
     23.01264      26.67311      27.54527
        Elite
     27.89474
>
```

The analysis reveals a significant correlation between rating class and wages. Elite players make an average of 150,453 EUR per week, which is 84 times more than Below Average players (1,791 EUR). The standard deviation of wages also goes up with rating class, with Elite players having the highest standard deviation (87,342 EUR). The mean age goes up slowly from 23 (Below Average) to 27.89 (Elite). This suggests that players with higher ratings are usually older because they need more experience to reach their best level of play.

Data Filtering::

Code:

```
data_filtered <- df %>% filter(age > 30)
cat("Players above age 30:", nrow(data_filtered), "\n")
head(data_filtered)
```

Output:

```
> data_filtered <- df %>% filter(age > 30)
> cat("Players above age 30:", nrow(data_filtered), "\n")
Players above age 30: 2823
> head(data_filtered)
      short_name player_positions overall
6  H. Çalhanoğlu          CDM, CM      86
10  A. Griezmann      ST, LM, CAM      85
17    R. De Paul     CM, CDM, CAM      84
22      M. Salah           RM, RW      91
23   M. Sabitzer          CDM, CM      80
25  João Cancelo       RB, LB, RM      84
   potential value_eur wage_eur age height_cm
6         86  47500000    61000  31       178
10        85  26000000   115000  34       176
17        84  34500000   100000  31       180
22        91  82000000   370000  33       175
23        80  15000000    52000  31       178
25        84  30500000    68000  31       182
   weight_kg preferred_foot pace shooting passing
6         69          Right   71       81      87
10        67           Left   77       86      85
17        70          Right   75       78      83
22        72           Left   89       88      86
23        76          Right   74       80      80
25        74          Right   83       73      84
   dribbling defending physic
6         82        81     73
10        88        59     73
17        83        76     83
22        90        45     76
23        80        76     76
25        84        78     74
>
```

There were 18,405 players in total, but only 2,823 were over 30 years old (about 15.3%). The filtered output shows that M. Salah (age 33, overall 91), A. Griezmann (age 34, overall 85), and H. Çalhanoğlu (age 31, overall 86) are all experienced veteran players. Most of these older players have high overall ratings, which shows that experienced professionals can still perform at a high level even as they get older.

Balancing the Dataset:

Creating a categorical target variable based on overall rating:

Code:

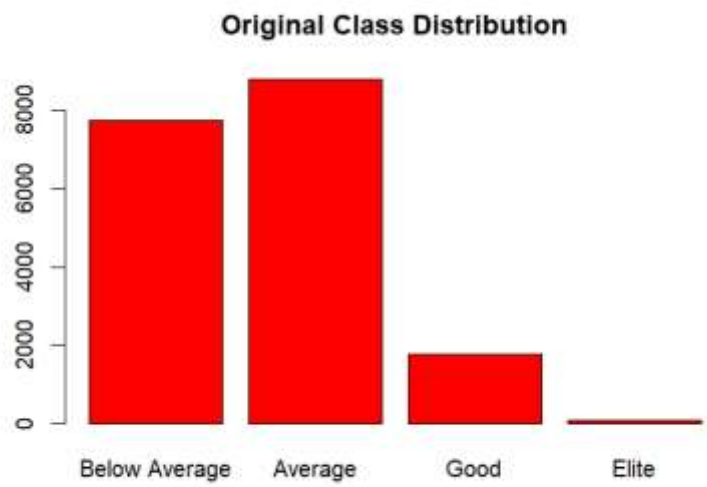
```
df$rating_class <- cut(df$overall,
                       breaks = c(0, 64, 74, 84, 100),
                       labels = c("Below Average", "Average", "Good", "Elite"))
idata <- df
```

Check original class distribution:

Code:

```
class_dist <- table(idata$rating_class)
class_dist
barplot(table(idata$rating_class), main = "Original Class Distribution", col = "red")
```

Output:



The bar chart and table show that the class distribution is very uneven. There are 8,792 players in the "Average" class (overall 65-74), and 7,751 players in the "Below Average" class (overall 0-64). Only 1,767 players are in the "Good" class (overall 75-84), and only 95 players are in the "Elite" class (overall 85-100). This big difference, where the biggest class is almost 93 times bigger than the smallest, could cause biases in predictive modeling. To fix this problem, balancing techniques will be used.

Balancing Using Over Sampling::

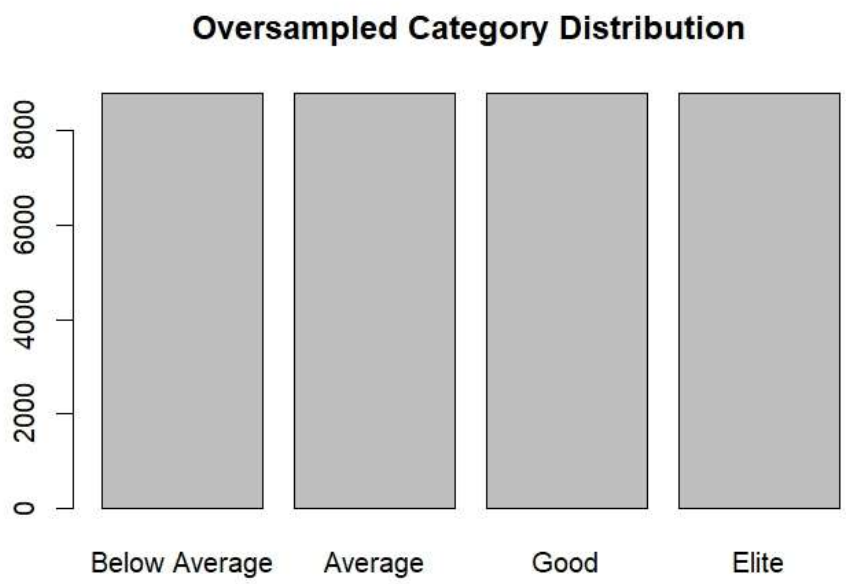
Duplicates minority-class samples to match the majority class size, creating a balanced dataset.

Code:

```
class_counts <- table(idata$rating_class)
max_count <- max(class_counts)
set.seed(100)
os_list <- lapply(names(class_counts), function(cl) {
  df <- filter(idata, rating_class == cl)
  if(nrow(df) < max_count) {
    df <- df %>% sample_n(max_count, replace = TRUE)
  }
  df
})

os_balanced_data <- bind_rows(os_list)
table(os_balanced_data$rating_class)
barplot(table(os_balanced_data$rating_class),
        main = "Oversampled Category Distribution")
```

Output:



```
> os_balanced_data <- bind_rows(os_list)
> table(os_balanced_data$rating_class)

Below Average       Average          Good
         8792          8792          8792
        Elite
         8792
> barplot(table(os_balanced_data$rating_class),
+         main = "Oversampled Category Distribution")
>
```

The oversampling method worked well to balance the dataset by making copies of samples from the minority class to match the size of the majority class, which was 8,792. There are now 8,792 players in each of the four classes: Below Average, Average, Good, and Elite. The "Elite" class, which started with only 95 players, was oversampled to 8,792 by randomly duplicating records of existing elite players with replacement. The barplot shows that all of the classes are now evenly spread out, which gets rid of the class imbalance that could lead to bias in predictive modeling.

Splitting the Dataset:

Code:

```
n <- nrow(df)
random_index <- sample(1:n, size = 0.8*n)
train_data <- df[random_index, ]
test_data <- df[-random_index, ]
cat("Training set rows:", nrow(train_data), "\n")
cat("Testing set rows:", nrow(test_data), "\n")
```

Output:

```
> n <- nrow(df)
> random_index <- sample(1:n, size = 0.8*n)
> train_data <- df[random_index, ]
> test_data <- df[-random_index, ]
> cat("Training set rows:", nrow(train_data), "\n")
Training set rows: 14724
> cat("Testing set rows:", nrow(test_data), "\n")
Testing set rows: 3681
>
```

We split the dataset into two parts: 80% for training and 20% for testing. There are 14,724 rows in the training set (80%) and 3,681 rows in the testing set (20%). The sample() function was used to randomly choose the rows so that the split was fair. The training set will be used to build the model and learn patterns, while the testing set will only be used to see how well the model works. This step makes sure that the data is split up correctly so that the model can be trained and tested.

Conclusion:

In this project, a complete data-science workflow has been implemented on the FC26 FIFA Player Dataset. Systematic cleaning of the raw data included missing values, noisy values, outliers, duplicate values, and non-uniform formats. These were truly important pre-processing measures to ensure the dataset was reliable, consistent as well as analytically valid. After cleaning, the resulting refined dataset was then used in exploratory data analysis where at any given time, descriptive statistics and visuals were used to offer valuable information on the distribution and behavior of important player values like overall rating, potential, age, and wages. Normalization, feature exploration, class balancing, and dataset splitting are subsequently used to further prepare data to be used in machine-learning tasks.

Overall, the project demonstrates that the organized data-science pipeline is significant between raw data and a clean and interpretable representation, to the analysis of underlying trends, up to the presentation of the data to a predictive model. The cleaned data is now much more organized, and sound, and it can be used to develop a model to add to the understanding and prediction of player performance and ratings.